

Java vs TypeScript

A Java developer's quick reference | Mathieu Neron | 2026

This cheatsheet anchors TypeScript on constructs you already know in Java. Left column: Java. Right column: TypeScript (5.x). The biggest mental shifts: **structural typing** (shape, not name), **single-threaded event loop** with `async/await`, **union/literal types**, and a vast **type system** that erases at runtime.

1. Hello world & program entry

Program entry point

Java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, " + args[0]);
    }
}
// javac Hello.java && java Hello world
```

TypeScript

```
// hello.ts (Node 20+ / Bun / Deno)
const name: string = process.argv[2];
console.log(`Hello, ${name}`);

// $ tsx hello.ts world (or: tsc && node hello.js)
// In a module, top-level code just runs at load.
```

File / module / package layout

Java

```
// One public class per file.
package com.acme;
// pom.xml / build.gradle declares deps.
```

TypeScript

```
// Each .ts file is a module if it has import/export.
// package.json + node_modules. Entry: "main" or "exports".
// Scripts via "scripts": { ... }; pnpm/npm/bun.
// tsconfig.json controls compiler.
```

2. Variables, constants, type inference

Local variables

Java

```
int x = 5;
final String s = "hi";
var list = new ArrayList<Integer>(); // Java 10+
```

TypeScript

```
let x: number = 5; // mutable
const s: string = "hi"; // immutable BINDING
const list: number[] = []; // type can be inferred
const arr = [1, 2, 3]; // inferred number[]
// `var` exists but avoid it - function-scoped, hoisted.
```

Const != deeply immutable

Java

```
final List<Integer> xs = new ArrayList<>(List.of(1,2));
xs.add(3); // OK - final binding only
List<Integer> immut = List.of(1, 2);
immut.add(3); // throws UnsupportedOperationException
```

TypeScript

```
const xs = [1, 2];
xs.push(3); // OK - const binding only
const immut: readonly number[] = [1, 2];
immut.push(3); // type error
const tuple = [1, 2] as const; // readonly tuple [1,2]
```

3. Primitive types & conversions

Numeric & boolean types

Java

```
byte/short/int/long; float/double;
int n = Integer.parseInt("42");
double x = Double.parseDouble("1.5");
String t = String.valueOf(42);
```

TypeScript

```
// number = 64-bit IEEE float (no separate int).
// bigint = arbitrary precision (literal: 42n).
const n: number = parseInt("42", 10);
const x: number = parseFloat("1.5");
const t: string = String(42);
const big: bigint = 42n;
```

Strings (template literals)

Java

```
String name = "world";
String s1 = "Hello, " + name;
String s2 = String.format("%s = %.2f", "pi", 3.14);
String body = ""
    Hello,
    World
    "";
```

TypeScript

```
const name = "world";
const s1 = `Hello, ${name}`;
const s2 = `${"pi"} = ${(3.14).toFixed(2)}`;
const body = `
    Hello,
    World
`;
// Tagged templates: fn`a ${x} b` -> fn(strings, x)
```

4. Control flow

If / else / ternary / nullish

Java

```
if (x > 0) { ... } else if (x == 0) { ... } else { ... }
int sign = x > 0 ? 1 : (x < 0 ? -1 : 0);

String name = (s != null) ? s : "default";
```

TypeScript

```
if (x > 0) { ... } else if (x === 0) { ... } else { ... }
const sign = x > 0 ? 1 : (x < 0 ? -1 : 0);

const name = s ?? "default"; // nullish-coalescing
const name2 = s || "default"; // FALSY (incl. "", 0)
const len = obj?.user?.name?.length; // optional chaining
```

Switch / discriminated unions

Java

```
String label = switch (shape) {
    case Circle c -> "circle r=" + c.r();
    case Rectangle(var w, var h) -> "rect " + w + "x" + h;
    default -> "unknown";
};
```

TypeScript

```
type Shape =
  | { kind: "circle"; r: number }
  | { kind: "rect"; w: number; h: number };

function label(s: Shape): string {
    switch (s.kind) {
        case "circle": return `circle r=${s.r}`;
        case "rect": return `rect ${s.w}x${s.h}`;
    }
    // Exhaustiveness: TS narrows `s` to never above.
}
```

5. Loops

Counting / iteration

Java

```
for (int i = 0; i < 10; i++) { ... }
for (var x : xs) { ... }
for (var e : map.entrySet()) {
    System.out.println(e.getKey() + "=" + e.getValue());
}
```

TypeScript

```
for (let i = 0; i < 10; i++) { ... }

for (const x of xs) { ... } // values
for (const [i, x] of xs.entries()) { ... }
for (const [k, v] of m) { ... } // Map
for (const k in obj) { ... } // KEYS (rarely)

xs.forEach((x, i) => { ... }); // higher-order
```

Async iteration

Java

```
// Reactive stack or a custom Iterator. No syntax sugar.
```

TypeScript

```
for await (const chunk of stream) { // AsyncIterable
    process(chunk);
}
```

6. Collections & data structures

Arrays, sets, maps

Java

```
List<Integer> xs = new ArrayList<>(List.of(1,2,3));
xs.add(4); xs.get(0); xs.size(); xs.remove(0);

Set<String> set = new HashSet<>(List.of("a","b"));
Map<String,Integer> m = new HashMap<>();
m.put("a", 1); m.getOrDefault("b", 0);
```

TypeScript

```
const xs: number[] = [1, 2, 3];
xs.push(4); xs[0]; xs.length; xs.shift();
xs.slice(1, 3); [...xs].reverse();

const set = new Set<string>(["a", "b"]);
set.add("c"); set.has("a"); set.delete("b");

const m = new Map<string, number>();
m.set("a", 1); m.get("a") ?? 0; // getOrDefault
// Plain object as record: { [k: string]: number }
const r: Record<string, number> = { a: 1 };
```

Tuples & destructuring

Java

```
record Pair<A,B>(A first, B second) {}
Pair<Integer,String> p = new Pair<>(1, "x");
int a = p.first(); String b = p.second();
```

TypeScript

```
const pair: [number, string] = [1, "x"];
const [a, b] = pair; // tuple destructure
const { x, y } = point; // object destructure
const { x: px, y: py } = point; // rename
const [head, ...rest] = xs; // rest pattern
```

7. Functions

Definition, default args, varargs

Java

```
int add(int a, int b) { return a + b; }
int add(int a) { return add(a, 1); } // overload
int sum(int... xs) { ... }
```

TypeScript

```
function add(a: number, b: number = 1): number {
  return a + b;
}
function sum(...xs: number[]): number {
  return xs.reduce((t, x) => t + x, 0);
}
// Function overloads via signatures + impl:
function fmt(x: number): string;
function fmt(x: Date): string;
function fmt(x: number | Date): string { ... }
```

Arrow functions / closures

Java

```
Function<Integer,Integer> sq = x -> x * x;
Predicate<String> nonEmpty = s -> !s.isEmpty();
list.forEach(System.out::println);
```

TypeScript

```
const sq = (x: number) => x * x;
const nonEmpty = (s: string) => s.length > 0;
xs.forEach((x) => console.log(x));
// Arrow funcs: lexical `this`. Use `function` for own `this`.
```

Streams / pipelines

Java

```
List<Integer> r = xs.stream()
  .filter(x -> x > 0)
  .map(x -> x * 2)
  .sorted()
  .toList();
int total = xs.stream().mapToInt(Integer::intValue).sum();
```

TypeScript

```
const r = xs.filter(x => x > 0)
  .map(x => x * 2)
  .sort((a, b) => a - b);
const total = xs.reduce((a, x) => a + x, 0);
```

8. Classes, interfaces, types

Class with fields, ctor, methods

Java

```
public class Point {
    private final double x, y;
    public Point(double x, double y) {
        this.x = x; this.y = y;
    }
    public double dist(Point o) {
        return Math.hypot(x - o.x, y - o.y);
    }
}
```

TypeScript

```
class Point {
    // Param-property shorthand assigns + declares fields.
    constructor(
        public readonly x: number,
        public readonly y: number,
    ) {}
    dist(o: Point): number {
        return Math.hypot(this.x - o.x, this.y - o.y);
    }
}
```

Inheritance, abstract, interface

Java

```
abstract class Shape {
    abstract double area();
}
interface Drawable { void draw(); }
class Circle extends Shape implements Drawable {
    private final double r;
    public Circle(double r) { this.r = r; }
    @Override double area() { return Math.PI*r*r; }
    @Override public void draw() { ... }
}
```

TypeScript

```
abstract class Shape {
    abstract area(): number;
}
interface Drawable { draw(): void; }
class Circle extends Shape implements Drawable {
    constructor(private readonly r: number) { super(); }
    area(): number { return Math.PI * this.r ** 2; }
    draw(): void { ... }
}
// Single-class inheritance + multiple interface impl.
```

Static, getters/setters

Java

```
public class Box {
    private int n;
    public int getN() { return n; }
    public void setN(int v) { n = v; }
    public static Box empty() { return new Box(); }
}
```

TypeScript

```
class Box {
    private n = 0;
    get value(): number { return this.n; }
    set value(v: number) { this.n = v; }
    static empty(): Box { return new Box(); }
}
const b = new Box(); b.value = 5; console.log(b.value);
```

9. Type system: structural typing

Sidebar — structural vs nominal

Java types are **nominal**: class A and class B with identical members are unrelated. TypeScript types are **structural**: shape matches structure, not name. This is the single biggest mental shift coming from Java — it powers a lot of TS's flexibility (and surprises).

interface vs type alias

Java

```
interface User { String name(); int age(); }
class Member implements User {
    public String name() { return "..."; }
    public int age() { return 0; }
}
// Must declare `implements`.
```

TypeScript

```
interface User { name: string; age: number; }
// or:
type User = { name: string; age: number; };

const m = { name: "x", age: 1, extra: true };
const u: User = m; // OK - has all required props

// `interface` is open (mergeable). `type` is closed.
// Use interface for object/class shapes, type for unions.
```

Unions, literals, narrowing

Java

```
// Closest analogs: sealed interfaces, enums.
sealed interface Status permits Active, Banned {}
record Active() implements Status {}
record Banned(String reason) implements Status {}
```

TypeScript

```
type Status = "active" | "banned" | "pending"; // literals
type Result<T> = { ok: true; value: T }
  | { ok: false; error: Error };

function handle(r: Result<string>) {
  if (r.ok) {
    r.value; // narrowed to { ok:true; value:string }
  } else {
    r.error; // narrowed
  }
}
```

Type guards & assertion

Java

```
if (s instanceof Member m) { use(m); }
// Pattern match (Java 16+)
```

TypeScript

```
if (typeof x === "string") { ... } // primitive
if (x instanceof Date) { ... } // class
if ("foo" in obj) { ... } // property test

function isUser(o: unknown): o is User { // user-defined
  return typeof o === "object" && o !== null
    && "name" in o && typeof (o as any).name === "string";
}

const v = data as User; // unsafe cast (avoid)
```

10. Generics / type parameters

Generic class & function

Java

```
class Box<T> {
  private final T value;
  public Box(T v) { this.value = v; }
  public T get() { return value; }
}

<T extends Comparable<T>> T max(T a, T b) {
  return a.compareTo(b) >= 0 ? a : b;
}

List<? extends Number> ns;
```

TypeScript

```
class Box<T> {
  constructor(public readonly value: T) {}
  get(): T { return this.value; }
}

function max<T extends { compareTo(o: T): number }>(
  a: T, b: T,
): T {
  return a.compareTo(b) >= 0 ? a : b;
}

// Bivariance & variance: TS doesn't model wildcards, but
// `readonly T[]` is roughly `List<? extends T>`.
```

Utility / mapped types

Java

```
// No equivalent. You hand-write each variant class.
```

TypeScript

```
type User = { name: string; age: number; email: string };
type PartialUser = Partial<User>; // all optional
type ReadonlyUser = Readonly<User>;
type Email = Pick<User, "email">;
type NoEmail = Omit<User, "email">;
type RequiredU = Required<PartialUser>;
type Keys = keyof User; // "name"|"age"|...
// Mapped types compile to runtime-erased types.
```

11. Error handling

Try / catch / finally

Java

```
try (var f = Files.newBufferedReader(p)) {  
    return f.readLine();  
} catch (IOException e) {  
    log.error("read failed", e);  
    throw new MyException("oops", e);  
} finally { cleanup(); }
```

TypeScript

```
try {  
    return await readFile(p, "utf8");  
} catch (e) {  
    // Caught value type is `unknown` (TS 4.4+).  
    if (e instanceof Error) {  
        log.error(`read failed: ${e.message}`);  
    }  
    throw new MyError("oops", { cause: e });  
} finally { cleanup(); }
```

Custom error

Java

```
public class NotFoundException extends RuntimeException {  
    public NotFoundException(String m) { super(m); }  
}  
throw new NotFoundException("user " + id);
```

TypeScript

```
class NotFoundError extends Error {  
    constructor(message: string) {  
        super(message);  
        this.name = "NotFoundError";  
    }  
}  
throw new NotFoundError(`user ${id}`);
```

12. Modules / packages / imports

Imports / exports

Java

```
package com.acme.svc;  
import java.util.List;  
import static com.acme.Utills.foo;
```

TypeScript

```
import { foo, bar } from "./utils";           // named  
import defaultExport from "./mod";           // default  
import * as utils from "./utils";            // namespace  
import type { User } from "./types";         // type-only  
  
export const x = 1;  
export function f() { ... }  
export default class C { ... }  
export type { User };  
// "./utils.js" extension required in NodeNext / ESM mode.
```

13. Concurrency: Promises & async/await

Async functions

Java

```
CompletableFuture<String> fut =  
    CompletableFuture.supplyAsync(() -> fetch())  
        .thenApply(String::trim)  
        .exceptionally(e -> "fallback");  
String r = fut.get();
```

TypeScript

```
async function run(): Promise<string> {  
    try {  
        const raw = await fetchData();  
        return raw.trim();  
    } catch {  
        return "fallback";  
    }  
}  
const r = await run();           // top-level await in modules
```

Java

```
CompletableFuture<Void> all = CompletableFuture.allOf(f1, f2);
CompletableFuture<Object> any = CompletableFuture.anyOf(f1, f2);
```

TypeScript

```
const [a, b] = await Promise.all([p1, p2]); // all OK
const r = await Promise.race([p1, p2]);
const settled = await Promise.allSettled([p1, p2]);
const first = await Promise.any([p1, p2]); // any FULFILL
```

Single-threaded event loop

TypeScript runs on JS engines, which are **single-threaded** by default. `async/await` schedules continuations on the event loop — no preemption, no shared-memory races on JS objects. For CPU-heavy work, spawn `Worker` threads (browser) or `worker_threads` (Node), which have isolated memory and communicate via `postMessage`. This is a fundamental shift from Java's preemptive threading + shared mutable state model.

14. Idiom appendix

Equality, null, undefined

Java

```
if (s == null) { ... } // identity
if (a.equals(b)) { ... } // value
String x = (s != null) ? s : "default";
```

TypeScript

```
if (s === null) { ... } // strict equality
if (s == null) { ... } // null OR undefined
// Always prefer === / !== - `==` does coercion.
const x = s ?? "default"; // null/undefined only
const y = s || "default"; // ALSO falsy: "", 0, false
```

Common gotchas (TS coming from Java)

- **Types erase at runtime.** `instanceof T` works only for classes; you can't `instanceof MyInterface`. Use type guards or a kind discriminator.
- **this binding.** Arrow functions capture `this` lexically; `function` expressions don't. Methods passed as callbacks lose `this` unless bound.
- **undefined vs null.** Both exist. Default uninitialized = `undefined`. Prefer one consistently; many codebases ban `null`.
- **== is coercion.** `0 == ""` is true. Always `===/!==`.
- **Array holes.** `[1,,3]` has a sparse slot; `map` skips it. Avoid.
- **any vs unknown.** `any` disables type-checking. `unknown` forces narrowing before use — prefer it.
- **strict mode.** Always set `"strict": true` in `tsconfig.json`; otherwise nullability checks are off.
- **Numbers are floats.** No int. `0.1 + 0.2 !== 0.3`. Use `bigint` for exact integers, libraries for decimal money math.
- **Tooling.** `tsc` (compiler), `tsx/ts-node` (run), `eslint`, `prettier`, `vitest/jest`, `tsd/expect-type` for type-level tests.